

4 – REPETITION

Repetition structures allow a block of code to run zero or more times. Without loops, many practical programs would be inefficient or outright impractical – for example, adding the numbers from one to one million. Loops are essential for processing collections, performing repeated calculations and automating repetitive tasks.

Python supports two main looping constructs:

- while loops
- for loops

Python does **not** support:

- do-while loops
- C-style for(init; condition; update) loops

Both loop types support the `break` and `continue` keywords.

4.a – While statements

The while loop performs **conditional iteration**. The statements in the loop body are processed **as long as** the condition is `True` and processing ceases when the assertion becomes `False`. It is useful when the number of iterations is not known in advance.

```
while «condition»:  
... # body of the loop
```

If the condition never becomes `False`, you will have created an infinite loop. Always ensure your loop has logic to eventually terminate.

```
>>> x=1  
>>> total =0  
>>> while x < 100:  
...     total += x
```

```
... x += 1
...
>>> print(total)
4950
```

4.b.1 – For loop

The for loop is a **determinate loop** i.e., the number of iterations is determined by an iterable object.

Common Iterables:

- strings
- lists
- tuples
- sets
- dictionaries
- range() object

4.b.1.1 – Iterating Over an Iterable

Iterables includes strings, lists, sets, tuples and dictionaries.

```
# list example
>>>a = [3, 5, 7, 9]
>>>for x in a:
...   print(x, end=' ')
...
3 5 7 9 >>>
```

```
# string example
>>> name = 'Narendra'
>>> for a in name:
...   print(a, end=' ')
...
```

```
N a r e n d r a >>>
```

```
fruits = ['apple', 'banana', 'orange']
for f in fruits:          #fruits is a list
    print(f)
```

4.b.1.2 – Using the range object

The range() function returns an immutable sequence of numbers. A for loop is normally used to iterate this sequence. The range function takes one mandatory and two optional arguments.

```
range([«start=0»], «end», [«step=1»])
```

```
>>>for x in range(3, 10, 2):
...   print(x, end=' ')
...
3, 5, 7, 9 >>>
```

4.b.1.2.1 – Using the _ (Discard Variable)

If the loop variable is not needed, use the _. This avoids linter warning and clearly signals intent.

```
# if you are not using the value, then the linter
# will complain about un-used variables. The under
# score (referred to as a 'discard') prevents this
```

```
# often annoying reporting.
>>>for _ in range( 10 ):
...   print('Hello', end=' ')
Hello Hello Hello Hello Hello Hello Hello Hello Hello Hello
```

4.b.1.3 – Using the Enumerate Function

The `enumerate()` function returns index-value pairs from a collection (a sequence or mapping).

```
enumerate(«collection», [«start=0»])
```

```
>>> profs='ilia yin arben narendra mehrdad'.split()
>>> #process all the items
... for pos, name in enumerate(profs):
...   print(f'{pos} {name}')
...
0 ilia
1 yin
2 arben
3 narendra
4 mehrdad
```

```
# you can start the count at a particular integer
>>> profs='ilia yin arben narendra mehrdad'.split()
>>> for pos, name in enumerate(profs, start=2):
...   print(f'{pos} {name}')
...
2 ilia
3 yin
```

```
4 arben
5 narendra
6 mehrdad
```

```
# you can iterate a dictionary
>>> profs = {'ilia': 'COMP304', 'yin': 'COMP306', 'arben': 'COMP305',
'narendra': 'COMP216', 'mehrdad': 'COMP225'}
>>> for pos, name in enumerate(profs):
...     print(f'{pos}: {name}')
...
0: ilia
1: yin
2: arben
3: narendra
4: mehrdad
```

```
# a better way to iterate a dictionary is to use the .items() method of the
dictionary class
>>>
>>> for prof, course in profs.items():
...     print(f'{prof.rjust(12)}: {course}')
...
      ilia: COMP304
      yin: COMP306
      arben: COMP305
narendra: COMP216
mehrdad: COMP225
```

4.b.1.4 – Nested Loop

A loop within another loop is referred to as a nested loop. The two loops (inner and outer) do not need to match.

```
distros = 'ubuntu mint centos'.split()
arch = '32-bit 64-bit'.split()
for d in distros:
    for a in arch:
        print(d, a)
    print('_____')
```

```
ubuntu 32-bit
ubuntu 64-bit
```

```
_____
mint 32-bit
mint 64-bit
```

```
_____
centos 32-bit
centos 64-bit
```

4.c – Loop control Keywords

4.c.1 – break

This keyword allows you to break free of the loop. You can use it in a for or while loop to terminate when a particular condition is detected.

```
scores = [34, 67, 99, 105]
for s in scores:
    if s > 100:
        print('Invalid')
        break
    print(s)
```

4.c.2 – continue

This keyword allows you to skip the rest of the statement in the loop body and go to the next. The loop will still continue, but it will start from the top of the loop statement.

```
n = 1
while n < 100:
    n += 1
    if n % 2 == 0:
        continue
    print(n)
#Prints all the odd numbers
```

4.c.3 – pass

This keyword allows you to place a nothing statement (one that does not do anything) where a statement is needed. It is often used when you are not sure or you have not decided on the implementation details. It can be used for the body of a class, a method, the body of a loop or a block in an if statement.

```
for x in range(20):
    if x % 3 == 0:
        pass          #a nothing statement
    else:
        print(x)
```

4.d – Infinite Loops

Sometimes, the condition you are checking to see if the loop should continue never becomes false. In this case, the loop will continue for eternity (or until your computer stops it). This is more common with while loops.

```
while 5 > 1:
    print("I'm a survivor")
```

4.d – Summary

This chapter introduces repetition structures, which allow a block of code to execute zero or more times and are essential for writing efficient and practical programs.

Python provides two looping constructs: the `while` loop, which performs conditional iteration when the number of repetitions is not known in advance, and the `for` loop, which performs determinate iteration over iterable objects such as strings, lists, dictionaries, and `range()` sequences. The chapter explains how to iterate over collections, generate numeric sequences using `range()`, track positions with `enumerate()`, and combine loops using nested structures. It also covers loop control keywords—`break`, `continue`, and `pass`—and highlights the importance of proper loop termination to avoid infinite loops.